ProClinic — Hospital Management System

Software Engineering DFD Documentation

Data Flow Models, Data Dictionary, Traceability Matrix & Module Structure Chart

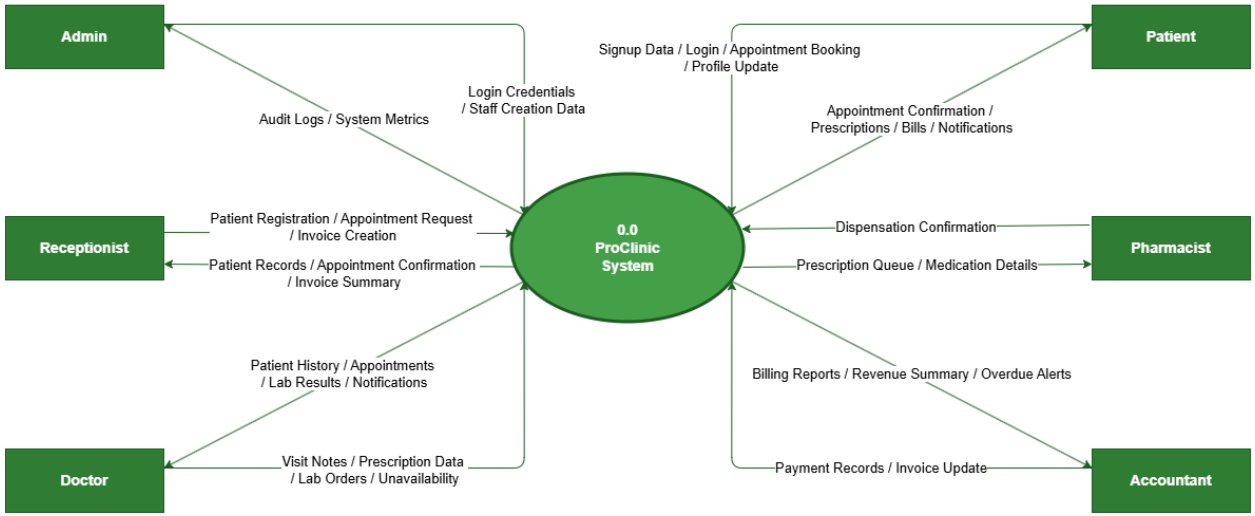
Prepared by:

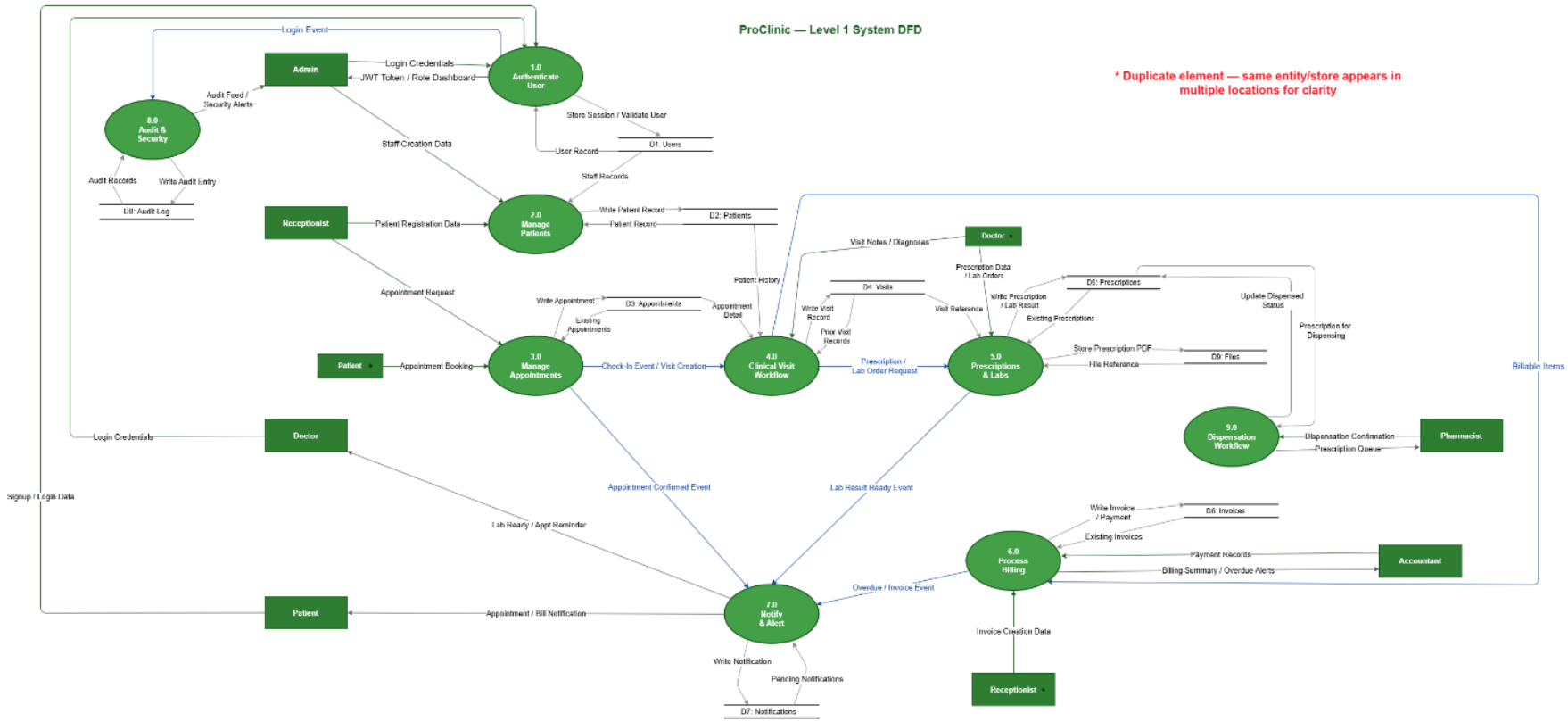
Student ID	Name	Role
202512006	Naitik Sutariya	Lead Developer
202512013	Harshal Prajapati	Backend Developer
202512090	Jaydip Chaudhari	UI/UX Developer
202512096	Parshwa Jain	Frontend Developer

Guided by: Prof. Jayprakash Lalchandani

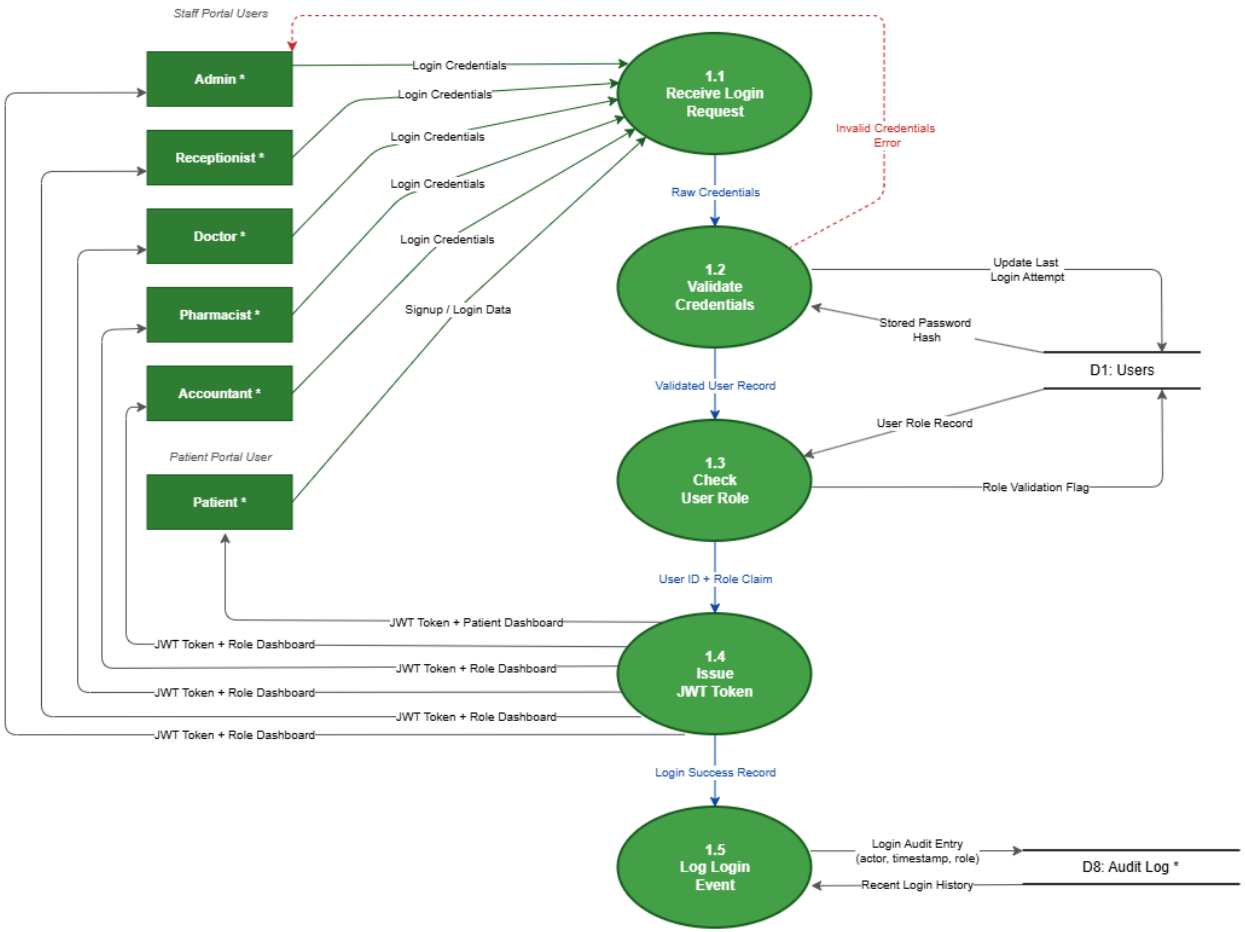
Subject: Software Engineering

ProClinic — Level 0 Context Diagram

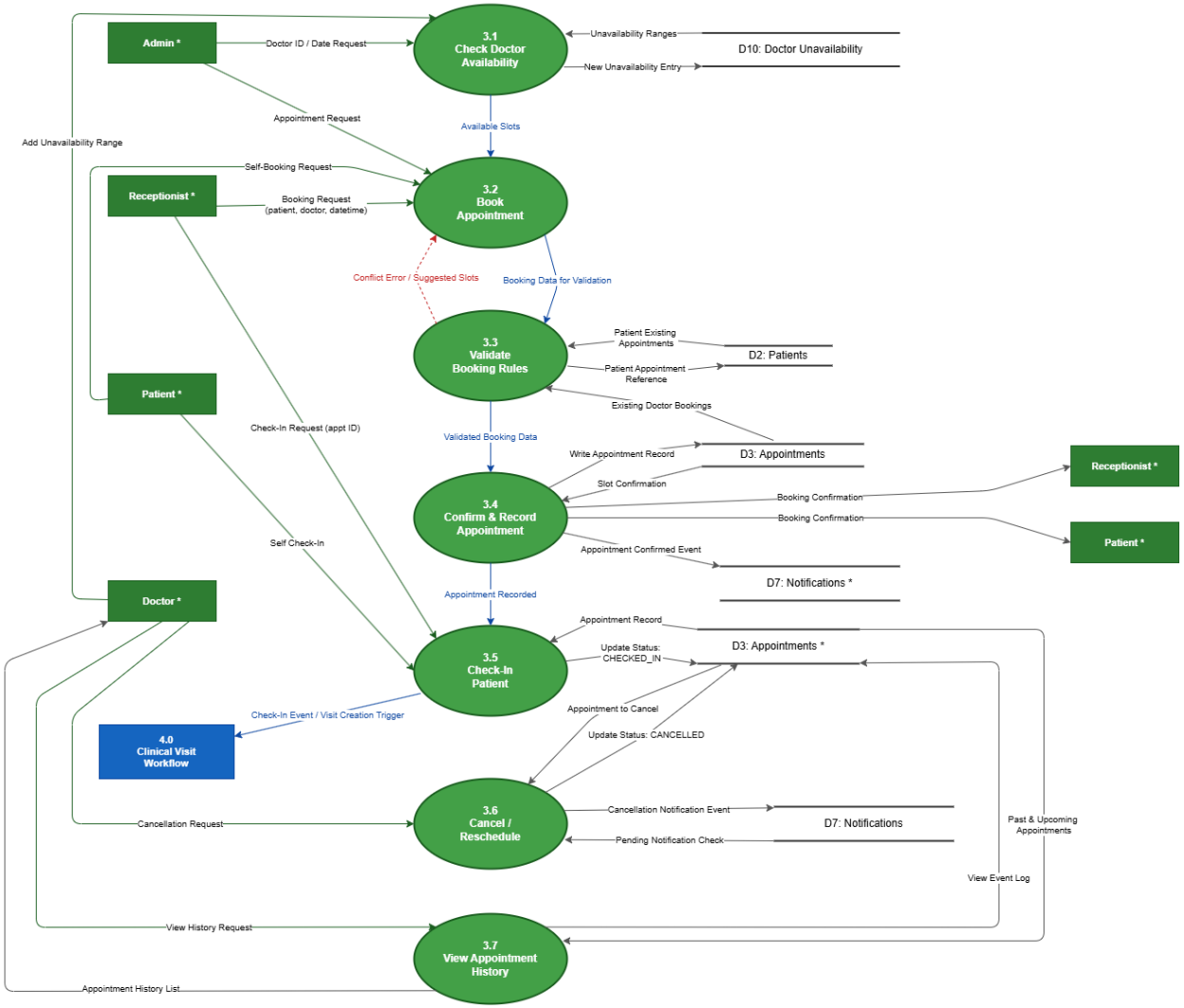



ProClinic — Level 1 System DFD
* Duplicate element — same entity/store appears in multiple locations for clarity
Login Event
Admin
Login Credentials
JWT Token / Role Dashboard
1.0 Authenticate User
Store Session / Validate User
D1: Users
User Record
Audit Feed / Security Alerts
8.0 Audit & Security
Audit Records
Write Audit Entry
D8: Audit Log
Staff Creation Data
Staff Records
Receptionist
Patient Registration Data
2.0 Manage Patients
Write Patient Record
Patient Record
D2: Patients
Doctor *
Visit Notes / Diagnoses
Patient History
Appointment Request
Write Appointment
D3: Appointments
Existing Appointments
Appointment Detail
D4: Visits
Write Visit Record
Prior Visit Records
Visit Reference
Prescription Data / Lab Orders
Write Prescription / Lab Result
Existing Prescriptions
D5: Prescriptions
Update Dispensed Status
Prescription for Dispensing
Patient *
Appointment Booking
3.0 Manage Appointments
Check-In Event / Visit Creation
4.0 Clinical Visit Workflow
Prescription / Lab Order Request
5.0 Prescriptions & Labs
Store Prescription PDF
File Reference
D9: Files
Billable Items
Login Credentials
Doctor
9.0 Dispensation Workflow
Dispensation Confirmation
Prescription Queue
Pharmacist
Signup / Login Data
Appointment Confirmed Event
Lab Result Ready Event
Lab Ready / Appt Reminder
Write Invoice / Payment
D6: Invoices
Existing Invoices
6.0 Process Billing
Payment Records
Billing Summary / Overdue Alerts
Accountant
Overdue / Invoice Event
Patient
Appointment / Bill Notification
7.0 Notify & Alert
Invoice Creation Data
Write Notification
Pending Notifications
D7: Notifications
Receptionist *

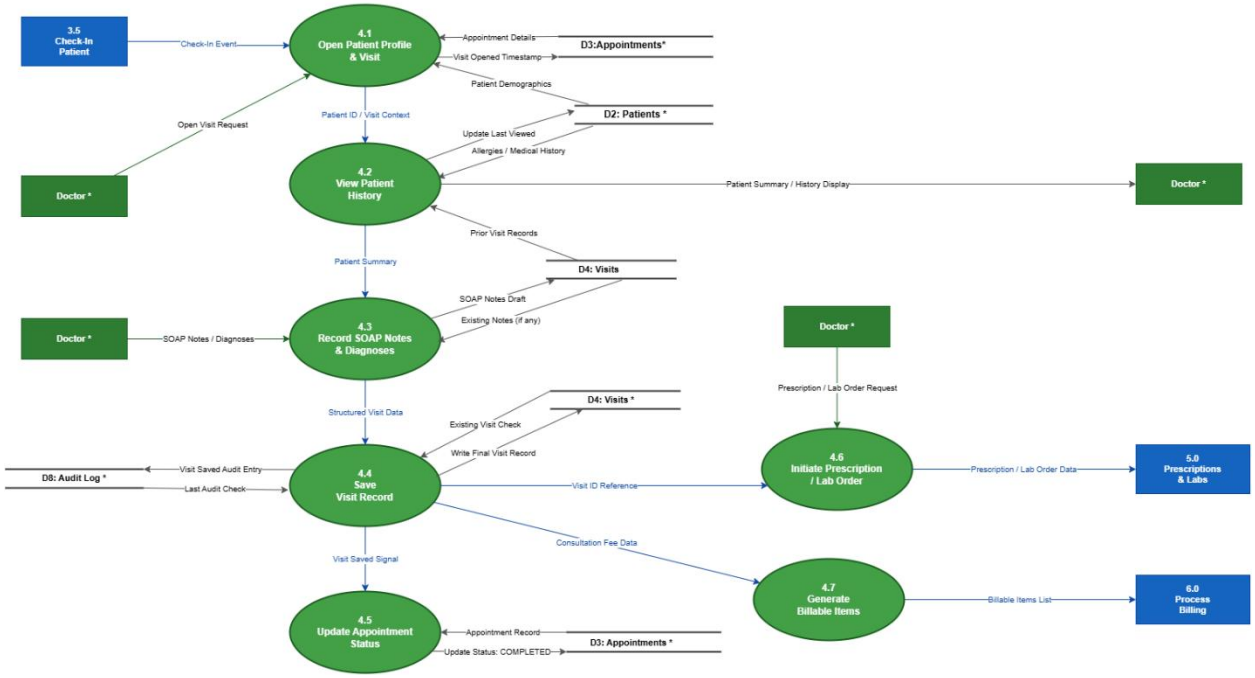
ProClinic — Level 2 DFD: Process 1.0 Authenticate User



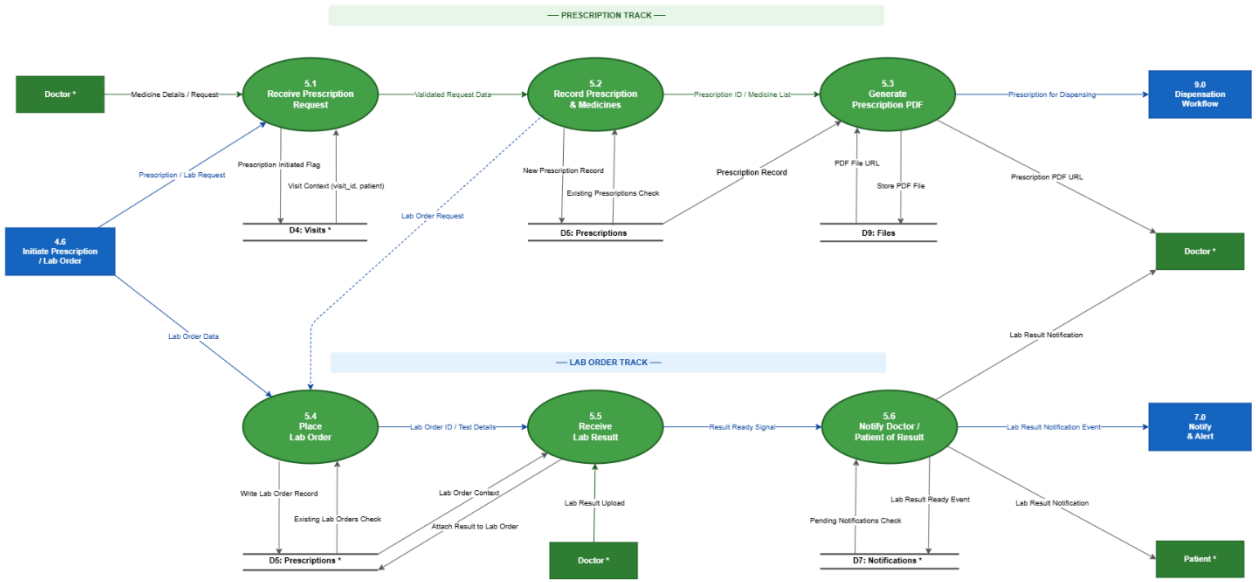
ProClinic — Level 2 DFD: Process 3.0 Manage Appointments



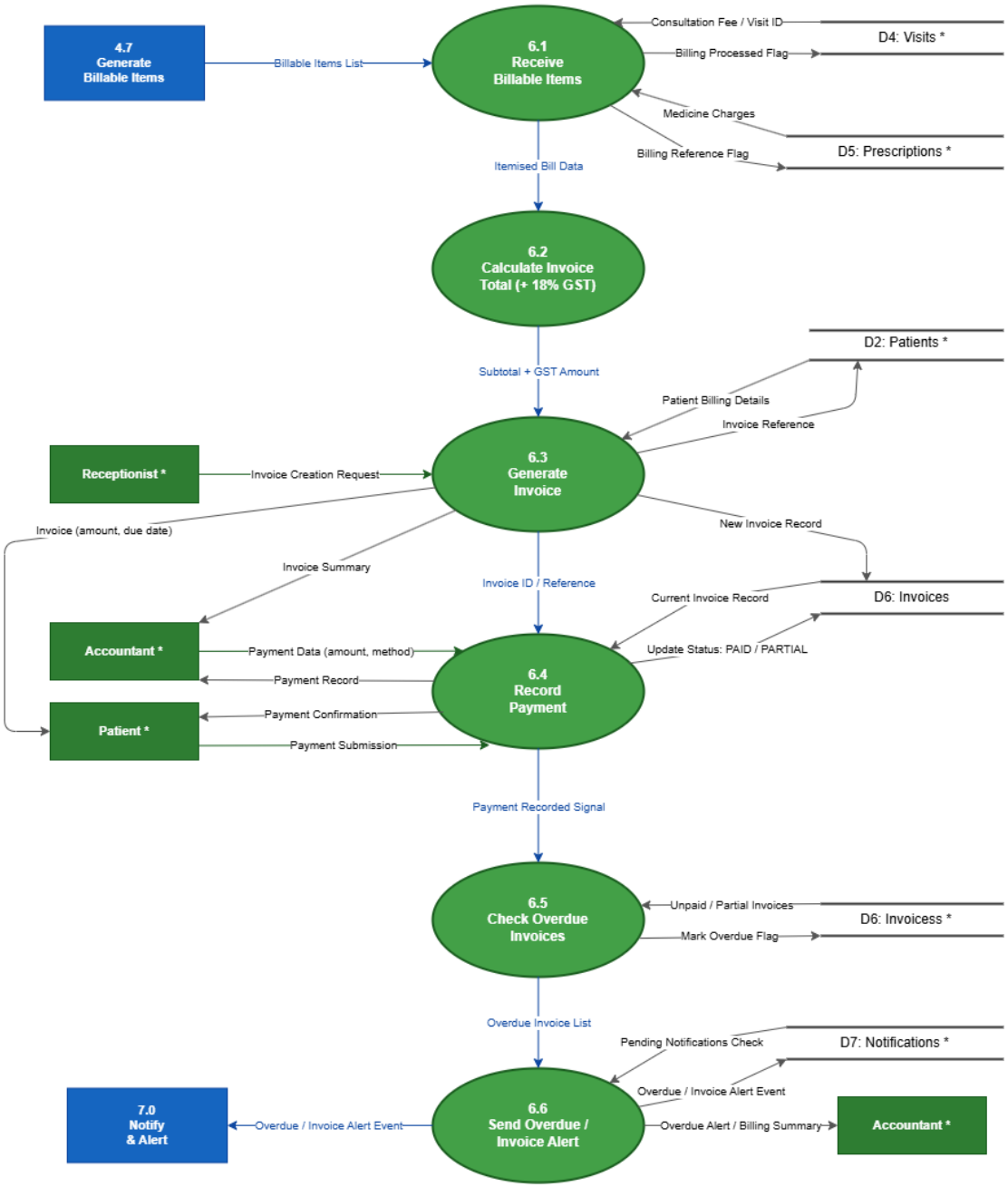
ProClinic — Level 2 DFD: Process 4.0 Clinical Visit Workflow



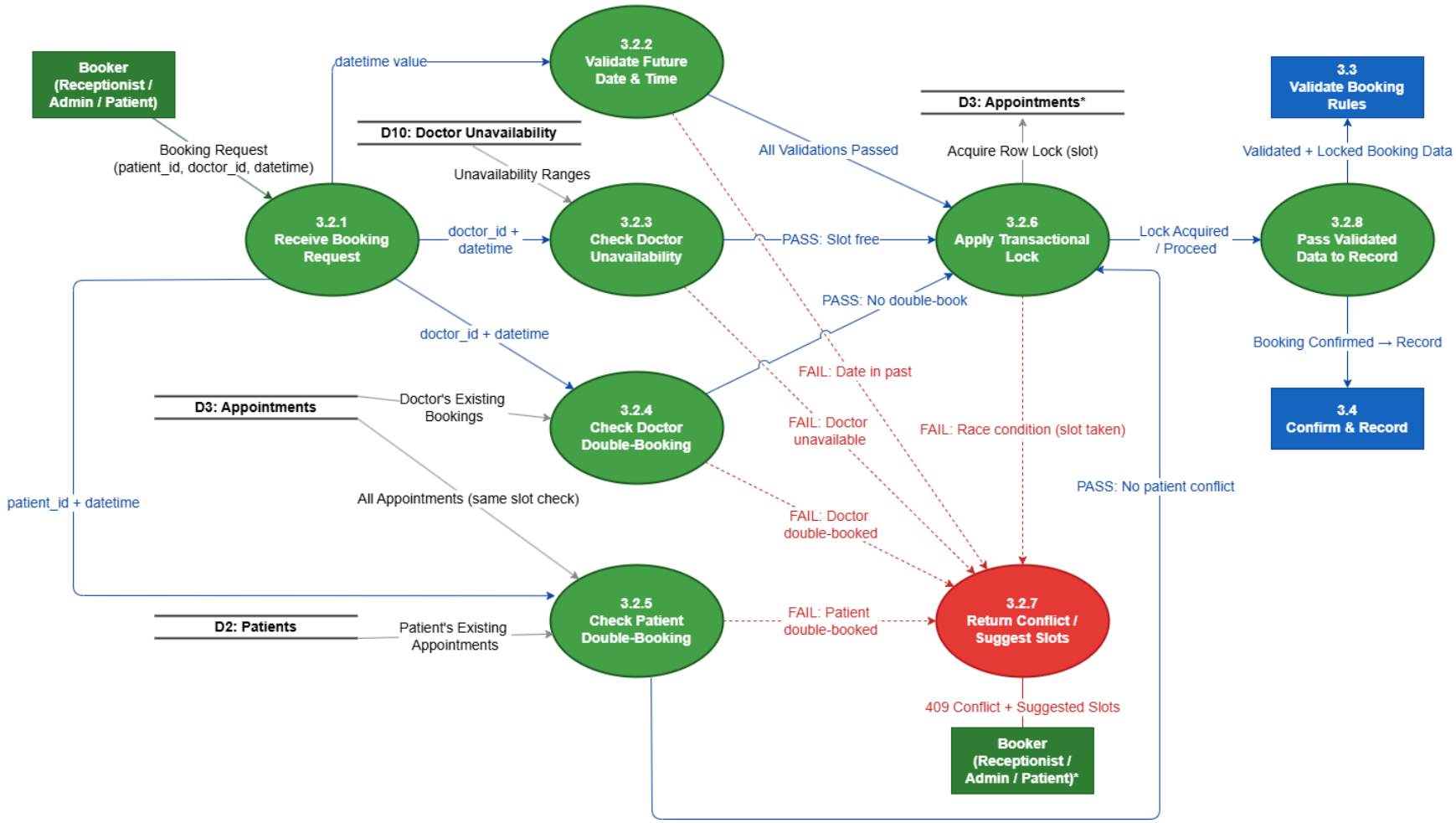
ProClinic — Level 2 DFD: Process 5.0 Prescriptions & Labs



ProClinic — Level 2 DFD: Process 6.0 Process Billing



ProClinic — Level 3 DFD: Process 3.2 Book Appointment



Data Dictionary

1. Introduction

This Data Dictionary defines all data elements used across the ProClinic Hospital Management System DFDs (Level 0, Level 1, and Level 2). It covers three categories of elements:

1. External Entities — actors outside the system boundary that interact with ProClinic.
2. Data Stores — persistent storage locations referenced in the DFDs (D1–D10).
3. Data Flows — named arrows between processes, entities, and stores in all DFD levels.

Every entry specifies the element name, type, a plain-language description, and its data composition (fields or structure). This dictionary is the single source of truth for all data flowing through ProClinic and serves as the traceability link between the SRS requirements and the DFD models.

2. External Entities

External entities are people or systems outside the ProClinic boundary that send data to or receive data from the system. They appear as filled rectangles in all DFD levels. Duplicate entities (marked with *) are the same actor placed in multiple diagram locations for readability.

Entity Name	Type	Description	Key Data Elements
Admin	External Entity	System administrator who manages staff accounts, views audit logs, system metrics, and has full access to all modules.	Username, Password, Role=Admin, Staff Creation Data, Audit Log Access

Entity Name	Type	Description	Key Data Elements
Receptionist	External Entity	Front-desk staff who registers patients, books appointments, performs check-in, and initiates invoice creation.	Username, Password, Role=Receptionist, Patient Registration Data, Appointment Request, Invoice Creation Request
Doctor	External Entity	Medical professional who views appointments, records SOAP notes, creates prescriptions, places lab orders, and sets unavailability.	Username, Password, Role=Doctor, Visit Notes, Prescription Data, Lab Orders, Unavailability Ranges
Patient	External Entity	Clinic patient who self-registers, books appointments, views prescriptions, and receives bills and notifications.	First Name, Last Name, Username, Password, DOB, Gender, Blood Group, Allergies, Address, Phone, Email
Pharmacist	External Entity	Pharmacy staff who views prescription queue and confirms dispensation of medicines.	Username, Password, Role=Pharmacist, Dispensation Confirmation
Accountant	External Entity	Finance staff who manages invoices, records payments, and views billing reports and overdue alerts.	Username, Password, Role=Accountant, Payment Data, Billing Summary

3. Data Stores

Data stores are repositories of persistent data used by the system. They appear as two horizontal parallel lines in all DFD levels. Each data store maps directly to one or more database tables in the ProClinic backend (Django/SQLite → PostgreSQL).

Store ID & Name	Type	Description	Key Fields / Schema
D1: Users	Data Store	Stores authentication credentials and role assignments for all system users (staff and patients).	user_id, username, password_hash (Argon2), email, role, last_login, account_status, two_factor_auth
D2: Patients	Data Store	Stores complete patient demographic, medical, and insurance information.	patient_id, first_name, last_name, dob, gender, blood_group, allergies, address, phone, email, emergency_contact, insurance_provider, policy_number, expiry_date
D3: Appointments	Data Store	Stores all appointment records including scheduling, status, and doctor-patient mapping.	appointment_id, patient_id, doctor_id, date, time, type, status (SCHEDULED/CHECKED_IN/COMPLETED/CANCELLED), created_by, created_at

Store ID & Name	Type	Description	Key Fields / Schema
D4: Visits	Data Store	Stores clinical visit records including SOAP notes, vitals, diagnoses, and visit metadata.	visit_id, appointment_id, patient_id, doctor_id, chief_complaint, diagnosis, soap_notes, vitals (BP, pulse, temp, weight), history, status, created_at
D5: Prescriptions	Data Store	Stores prescription records with multi-medicine line items, lab orders, and results.	prescription_id, visit_id, patient_id, doctor_id, medicine_name, dosage, frequency, duration, lab_order_id, lab_result, result_file, status (ACTIVE/DISPENSED), created_at
D6: Invoices	Data Store	Stores invoice records with line items, GST calculation, payment status, and payment history.	invoice_id, patient_id, visit_id, line_items[], subtotal, gst_rate (18%), gst_amount, total, status (UNPAID/PARTIAL/PAID), due_date, overdue_flag, created_at
D7: Notifications	Data Store	Stores pending and sent	notification_id, type (APPOINTMENT/LAB/BILL/OVERDUE), recipient_id,

Store ID & Name	Type	Description	Key Fields / Schema
		notification events for appointments, lab results, bills, and reminders.	message, channel (EMAIL/SMS/PUSH), status (PENDING/SENT), created_at
D8: Audit Log	Data Store	Stores system-wide audit trail of all administrative and clinical actions.	audit_id, actor_id, actor_role, action (CREATE/UPDATE/DELETE/LOGIN), target_table, target_id, timestamp, ip_address
D9: Files	Data Store	Stores uploaded files including prescription PDFs, lab result documents, and research papers.	file_id, entity_type, entity_id, file_name, file_path, mime_type, file_size, uploaded_by, uploaded_at
D10: Doctor Unavailability	Data Store	Stores date-time ranges when doctors are marked	unavailability_id, doctor_id, start_datetime, end_datetime, reason, created_at

Store ID & Name	Type	Description	Key Fields / Schema
		unavailable for appointments.	

4. Data Flows

Data flows are named arrows between elements in the DFDs. Each flow carries a specific, named set of data. The table below lists every flow by name, its source and destination, and the exact fields it carries. Flows are grouped by the process they primarily belong to.

4.1 P1.0 Authenticate User flows

Covers login credential submission from all six user types, credential validation against D1: Users using Argon2 hash comparison, user role resolution, JWT token issuance with role claim, error response for invalid credentials, and login event writing to D8: Audit Log.

Flow Name	Type	Source → Destination	Composition / Fields
Login Credentials	*Flow*	Admin / Doctor / Receptionist / Pharmacist / Accountant → P1.1	username: String, password: String, portal_type: {staff|patient}
Signup / Login Data	*Flow*	Patient → P1.1	first_name, last_name, username, email, password, dob, gender, phone

Flow Name	Type	Source → Destination	Composition / Fields
Raw Credentials	*Flow*	P1.1 → P1.2	username: String, password_plaintext: String
Stored Password Hash	*Flow*	D1: Users → P1.2	user_id, password_hash: String (Argon2), account_status
Update Last Login Attempt	*Flow*	P1.2 → D1: Users	user_id, last_attempt_at: DateTime, attempt_status: {success|fail}
Invalid Credentials Error	*Flow*	P1.2 → User Entity	error_code: 401, message: String
Validated User Record	*Flow*	P1.2 → P1.3	user_id, username, account_status
User Role Record	*Flow*	D1: Users → P1.3	user_id, role: Enum{Admin|Doctor|Receptionist|Pharmacist|Accountant|Patient}
Role Validation Flag	*Flow*	P1.3 → D1: Users	user_id, role_verified: Boolean, verified_at: DateTime
User ID + Role Claim	*Flow*	P1.3 → P1.4	user_id, role, permissions[]

Flow Name	Type	Source → Destination	Composition / Fields
JWT Token + Role Dashboard	*Flow*	P1.4 → Staff Entities	jwt_token: String, refresh_token: String, role: String, dashboard_url: String, expires_at: DateTime
JWT Token + Patient Dashboard	*Flow*	P1.4 → Patient	jwt_token: String, refresh_token: String, patient_id: String, dashboard_url: String
Login Success Record	*Flow*	P1.4 → P1.5	user_id, role, login_at: DateTime, ip_address: String
Login Audit Entry	*Flow*	P1.5 → D8: Audit Log	actor_id, actor_role, action: LOGIN, timestamp: DateTime, ip_address
Recent Login History	*Flow*	D8: Audit Log → P1.5	actor_id, recent_logins[]: {timestamp, ip, status}

4.2 P2.0 Manage Patients & P3.0 Manage Appointments flows

Covers patient registration by receptionist and patient record retrieval (P2.0). Also covers doctor availability checking against D10, appointment booking with conflict validation for both doctor and patient, booking confirmation, check-in triggering that hands off to P4.0, cancellation and rescheduling with notifications written to D7, and appointment history viewing (P3.0).

Flow Name	Type	Source → Destination	Composition / Fields
Patient Registration Data	*Flow*	Receptionist → P2.0	first_name, last_name, dob, gender, blood_group, allergies, address, phone, email, emergency_contact, insurance_info
Write Patient Record	*Flow*	P2.0 → D2: Patients	All patient fields (see D2 schema in Section 3)
Patient Record	*Flow*	D2: Patients → P2.0	patient_id, full patient record
Doctor ID / Date Request	*Flow*	Admin → P3.1	doctor_id, date: Date, action: check_availability
Add Unavailability Range	*Flow*	Admin → P3.1	doctor_id, start_datetime: DateTime, end_datetime: DateTime, reason: String
Unavailability Ranges	*Flow*	D10: Doctor Unavailability → P3.1	unavailability_id, doctor_id, start_datetime, end_datetime
New Unavailability Entry	*Flow*	P3.1 → D10: Doctor Unavailability	doctor_id, start_datetime, end_datetime, reason, created_at
Available Slots	*Flow*	P3.1 → P3.2	doctor_id, date: Date, available_times[]: Time
Appointment Request	*Flow*	Admin / Receptionist / Patient → P3.2	patient_id, doctor_id, requested_date: Date, requested_time: Time, type: String

Flow Name	Type	Source → Destination	Composition / Fields
Booking Data for Validation	*Flow*	P3.2 → P3.3	patient_id, doctor_id, date, time, type
Patient Existing Appointments	*Flow*	D2: Patients → P3.3	patient_id, existing_appointment_ids[], dates_times[]
Patient Appointment Reference	*Flow*	D2: Patients → P3.3	patient_id, patient_appointment_ref for conflict check
Existing Doctor Bookings	*Flow*	D3: Appointments → P3.3	doctor_id, booked_slots[]: {date, time, appointment_id}
Conflict Error / Suggested Slots	*Flow*	P3.3 → P3.2	error_code: 409, conflict_reason: String, suggested_slots[]: DateTime
Validated Booking Data	*Flow*	P3.3 → P3.4	patient_id, doctor_id, date, time, type, validation_passed: true
Write Appointment Record	*Flow*	P3.4 → D3: Appointments	appointment_id (generated), patient_id, doctor_id, date, time, status: SCHEDULED
Slot Confirmation	*Flow*	D3: Appointments → P3.4	appointment_id, slot confirmed: Boolean
Booking Confirmation	*Flow*	P3.4 → Receptionist* / Patient*	appointment_id, doctor_name, date, time, consultation_fee

Flow Name	Type	Source → Destination	Composition / Fields
Appointment Confirmed Event	*Flow*	P3.4 → D7: Notifications	notification_type: APPOINTMENT, recipient_id, appointment_id, message: String
Appointment Record	*Flow*	D3: Appointments → P3.5	appointment_id, patient_id, doctor_id, date, time, status
Update Status: CHECKED_IN	*Flow*	P3.5 → D3: Appointments*	appointment_id, status: CHECKED_IN, checked_in_at: DateTime
Check-In Event / Visit Creation	*Flow*	P3.5 → P4.0 Clinical Visit Workflow	appointment_id, patient_id, doctor_id, check_in_at: DateTime
Appointment to Cancel	*Flow*	P3.6 → D3: Appointments*	appointment_id, action: cancel
Update Status: CANCELLED	*Flow*	P3.6 → D3: Appointments*	appointment_id, status: CANCELLED, cancelled_at: DateTime, reason: String
Cancellation Notification Event	*Flow*	P3.6 → D7: Notifications	notification_type: CANCELLATION, recipient_id, appointment_id, reason: String
Pending Notification Check	*Flow*	D7: Notifications → P3.6	existing_notifications[]: {notification_id, appointment_id} to prevent duplicates
Past & Upcoming Appointments	*Flow*	D3: Appointments → P3.7	appointments[]: {appointment_id, date, time, patient_name, type, status}

Flow Name	Type	Source → Destination	Composition / Fields
View Event Log	*Flow*	D3: Appointments → P3.7	appointment audit history for the requesting user
View History Request	*Flow*	Doctor* → P3.7	doctor_id, filter: {date_range, status}
Appointment History List	*Flow*	P3.7 → Doctor*	appointments[]: {appointment_id, patient_name, date, time, type, status}

4.3 P4.0 Clinical Visit Workflow flows

Covers doctor-initiated visit opening by reading D3: Appointments and D2: Patients, full patient history retrieval from D4: Visits, SOAP notes and diagnosis entry, final visit record saving to D4 with duplicate prevention, audit entry writing to D8, appointment status update to COMPLETED in D3, and trigger flows to P5.0 (prescriptions and labs) and P6.0 (billing).

Flow Name	Type	Source → Destination	Composition / Fields
Open Visit Request	*Flow*	Doctor → P4.1	appointment_id, doctor_id (from JWT), action: open_visit
Appointment Details	*Flow*	D3: Appointments* → P4.1	appointment_id, patient_id, doctor_id, date, time, type, status
Visit Opened Timestamp	*Flow*	P4.1 → D3: Appointments*	appointment_id, visit_opened_at: DateTime

Flow Name	Type	Source → Destination	Composition / Fields
Patient Demographics	*Flow*	D2: Patients* → P4.1	patient_id, name, dob, gender, blood_group, allergies
Patient ID / Visit Context	*Flow*	P4.1 → P4.2	patient_id, appointment_id, visit_context: {allergies, current_medications}
Update Last Viewed	*Flow*	P4.2 → D2: Patients*	patient_id, last_viewed_by: doctor_id, last_viewed_at: DateTime
Allergies / Medical History	*Flow*	D2: Patients* → P4.2	patient_id, known_allergies[], medical_history: String
Prior Visit Records	*Flow*	D4: Visits → P4.2	visits[]: {visit_id, date, diagnosis, doctor_name, soap_notes}
Patient Summary / History	*Flow*	P4.2 → Doctor*	allergy_summary, prior_diagnoses[], recent_medications[], last_visit_date
Patient Summary	*Flow*	P4.2 → P4.3	patient_id, allergy_summary, prior_diagnoses[], recent_medications[]
SOAP Notes / Diagnoses	*Flow*	Doctor* → P4.3	chief_complaint, subjective, objective, assessment, plan, diagnosis, vitals: {bp, pulse, temp, weight}
SOAP Notes Draft	*Flow*	P4.3 → D4: Visits	visit_id (draft), soap_fields, saved_at: DateTime (auto-save)

Flow Name	Type	Source → Destination	Composition / Fields
Existing Notes (if any)	*Flow*	D4: Visits → P4.3	visit_id, existing_draft_notes (for resume editing)
Structured Visit Data	*Flow*	P4.3 → P4.4	visit_id (new), appointment_id, patient_id, doctor_id, all SOAP fields, vitals, diagnosis
Existing Visit Check	*Flow*	P4.4 → D4: Visits*	appointment_id, check: does_visit_exist — prevents duplicate
Write Final Visit Record	*Flow*	P4.4 → D4: Visits*	All Visit fields (see D4 schema in Section 3)
Visit Saved Audit Entry	*Flow*	P4.4 → D8: Audit Log*	actor_id, action: CREATE, target_table: visits, target_id: visit_id, timestamp
Last Audit Check	*Flow*	D8: Audit Log* → P4.4	actor_id, recent_audit_entries[] — duplicate save guard
Visit Saved Signal	*Flow*	P4.4 → P4.5	appointment_id, visit_id, saved_at: DateTime
Appointment Record	*Flow*	D3: Appointments* → P4.5	appointment_id, current_status
Update Status: COMPLETED	*Flow*	P4.5 → D3: Appointments*	appointment_id, status: COMPLETED, completed_at: DateTime
Visit ID Reference	*Flow*	P4.4 → P4.6	visit_id, patient_id, doctor_id

Flow Name	Type	Source → Destination	Composition / Fields
Prescription / Lab Order	*Flow*	Doctor* → P4.6	request_type: {prescription|lab|both}, medicines[], test_names[]
Prescription / Lab Order Data	*Flow*	P4.6 → P5.0 Prescriptions & Labs	visit_id, patient_id, doctor_id, request_type, medicines[], test_names[]
Consultation Fee Data	*Flow*	P4.4 → P4.7	visit_id, consultation_fee: Decimal, appointment_type: String
Billable Items List	*Flow*	P4.7 → P6.0 Process Billing	visit_id, items[]: {description, quantity, unit_price}, consultation_fee: Decimal

4.4 P5.0 Prescriptions & Labs flows

Covers prescription request validation against D4: Visits, multi-medicine prescription record creation in D5, duplicate prescription prevention, prescription PDF generation stored in D9: Files, lab order placement and persistence in D5, lab result upload and attachment to the order record, and result-ready notifications written to D7 with dispatch to both doctor and patient.

Flow Name	Type	Source → Destination	Composition / Fields
Medicine Details / Request	*Flow*	Doctor* → P5.1	medicine_name: String, dosage: String, frequency: String, duration: String, quantity: Integer
Prescription / Lab Request	*Flow*	P4.6 → P5.1	visit_id, patient_id, doctor_id, request_type: {prescription|lab|both}

Flow Name	Type	Source → Destination	Composition / Fields
Prescription Initiated Flag	*Flow*	P5.1 → D4: Visits*	visit_id, prescription_initiated: Boolean, initiated_at: DateTime
Visit Context (visit_id, patient)	*Flow*	D4: Visits* → P5.1	visit_id, patient_id, doctor_id, diagnosis — validates the request belongs to an open visit
Validated Request Data	*Flow*	P5.1 → P5.2	visit_id, patient_id, medicines[], request_type
New Prescription Record	*Flow*	P5.2 → D5: Prescriptions	prescription_id (new), visit_id, patient_id, doctor_id, medicines[], status: ACTIVE, created_at
Existing Prescriptions Check	*Flow*	D5: Prescriptions → P5.2	visit_id, existing_prescription_id (if any) — duplicate guard
Lab Order Request	*Flow*	P5.2 → P5.4	visit_id, patient_id, doctor_id, test_names[], urgency: String
Prescription ID / Medicine List	*Flow*	P5.2 → P5.3	prescription_id, patient_name, doctor_name, medicines[], date
Prescription Record	*Flow*	D5: Prescriptions → P5.3	Full prescription record for PDF template rendering

Flow Name	Type	Source → Destination	Composition / Fields
Store PDF File	*Flow*	P5.3 → D9: Files	file_id (new), entity_type: prescription, entity_id: prescription_id, file_path, mime_type: application/pdf
PDF File URL	*Flow*	D9: Files → P5.3	file_path: String, download_url: String
Prescription for Dispensing	*Flow*	P5.3 → P9.0 Dispensation Workflow	prescription_id, patient_name, medicines[], status: ACTIVE
Prescription PDF URL	*Flow*	P5.3 → Doctor*	prescription_id, download_url: String
Write Lab Order Record	*Flow*	P5.4 → D5: Prescriptions*	lab_order_id (new), visit_id, test_names[], status: PENDING, ordered_at
Existing Lab Orders Check	*Flow*	D5: Prescriptions* → P5.4	visit_id, existing_lab_orders[] — duplicate guard
Lab Order ID / Test Details	*Flow*	P5.4 → P5.5	lab_order_id, test_names[], patient_id
Lab Order Context	*Flow*	D5: Prescriptions* → P5.5	lab_order_id, visit_id, patient_id, test_names[]
Lab Result Upload	*Flow*	Doctor* → P5.5	lab_order_id, result_values{}, result_file (optional), result_date: Date

Flow Name	Type	Source → Destination	Composition / Fields
Attach Result to Lab Order	*Flow*	P5.5 → D5: Prescriptions*	lab_order_id, result_data, result_file_id, status: COMPLETED, resulted_at: DateTime
Result Ready Signal	*Flow*	P5.5 → P5.6	lab_order_id, patient_id, doctor_id, result_summary: String
Lab Result Ready Event	*Flow*	P5.6 → D7: Notifications*	notification_type: LAB_RESULT, recipient_ids[], lab_order_id, message: String
Pending Notifications Check	*Flow*	D7: Notifications* → P5.6	existing_alerts[]: {notification_id, lab_order_id} to prevent duplicate sends
Lab Result Notification	*Flow*	P5.6 → Doctor* / Patient*	lab_order_id, test_names[], result_summary: String, download_url: String
Lab Result Notification Event	*Flow*	P5.6 → P7.0 Notify & Alert	notification_type: LAB_RESULT, recipient_ids[], lab_order_id

4.5 P6.0 Process Billing flows

Covers collection of billable items from D4: Visits and D5: Prescriptions, 18% GST calculation, invoice generation with patient billing details written to D6: Invoices, payment recording with PAID/PARTIAL status updates, overdue invoice detection and flagging in D6, and overdue alert dispatch written to D7: Notifications with summary sent to the Accountant.

Flow Name	Type	Source → Destination	Composition / Fields
Billable Items List	*Flow*	P4.7 → P6.1	visit_id, items[]: {description, quantity, unit_price}, consultation_fee: Decimal
Consultation Fee / Visit ID	*Flow*	D4: Visits* → P6.1	visit_id, consultation_fee: Decimal
Billing Processed Flag	*Flow*	P6.1 → D4: Visits*	visit_id, billing_initiated: Boolean
Medicine Charges	*Flow*	D5: Prescriptions* → P6.1	prescription_id, medicines[]: {name, quantity, unit_price}
Billing Reference Flag	*Flow*	P6.1 → D5: Prescriptions*	prescription_id, billing_reference_set: Boolean
Itemised Bill Data	*Flow*	P6.1 → P6.2	items[]: {description, quantity, unit_price, line_total}, subtotal: Decimal
Subtotal + GST Amount	*Flow*	P6.2 → P6.3	subtotal: Decimal, gst_rate: 18%, gst_amount: Decimal, total: Decimal
Invoice Creation Request	*Flow*	Receptionist* → P6.3	patient_id, visit_id, action: generate_invoice
Patient Billing Details	*Flow*	D2: Patients* → P6.3	patient_id, name, address, insurance_provider, policy_number
Invoice Reference	*Flow*	P6.3 → D2: Patients*	patient_id, latest_invoice_id (reference stored on patient record)

Flow Name	Type	Source → Destination	Composition / Fields
New Invoice Record	*Flow*	P6.3 → D6: Invoices	invoice_id (new), patient_id, visit_id, line_items[], subtotal, gst, total, status: UNPAID, due_date
Invoice (amount, due date)	*Flow*	P6.3 → Patient*	invoice_id, total_amount, due_date, payment_instructions
Invoice Summary	*Flow*	P6.3 → Accountant*	invoice_id, patient_name, total, status, created_at
Invoice ID / Reference	*Flow*	P6.3 → P6.4	invoice_id, total: Decimal, due_date: Date
Current Invoice Record	*Flow*	D6: Invoices → P6.4	invoice_id, total, paid_amount, status, due_date
Payment Data (amount, method)	*Flow*	Accountant* → P6.4	invoice_id, amount_paid: Decimal, payment_method: {cash|card|UPI}, payment_date: Date
Payment Submission	*Flow*	Patient* → P6.4	invoice_id, amount_paid: Decimal, payment_method, transaction_ref: String
Update Status: PAID / PARTIAL	*Flow*	P6.4 → D6: Invoices	invoice_id, paid_amount, status: {PAID|PARTIAL}, paid_at: DateTime

Flow Name	Type	Source → Destination	Composition / Fields
Payment Confirmation	*Flow*	P6.4 → Patient*	invoice_id, amount_paid, remaining_balance, status, receipt_number
Payment Record	*Flow*	P6.4 → Accountant*	invoice_id, payment_amount, method, paid_at, updated_status
Payment Recorded Signal	*Flow*	P6.4 → P6.5	invoice_id, payment_status: {PAID|PARTIAL}
Unpaid / Partial Invoices	*Flow*	D6: Invoices → P6.5	invoices[]: {invoice_id, patient_id, total, paid_amount, due_date, status}
Mark Overdue Flag	*Flow*	P6.5 → D6: Invoices*	invoice_id, overdue_flag: true, overdue_since: DateTime
Overdue Invoice List	*Flow*	P6.5 → P6.6	invoices[]: {invoice_id, patient_name, total, due_date, overdue_days: Integer}
Pending Notifications Check	*Flow*	D7: Notifications* → P6.6	existing_alerts[]: {notification_id, invoice_id} to prevent duplicate sends
Overdue / Invoice Alert Event	*Flow*	P6.6 → D7: Notifications*	notification_type: OVERDUE, recipient_ids[], invoice_ids[], message: String
Overdue Alert / Billing Summary	*Flow*	P6.6 → Accountant*	overdue_count: Integer, total_overdue_amount: Decimal, invoice_list[]

Flow Name	Type	Source → Destination	Composition / Fields
Overdue / Invoice Alert Event	*Flow*	P6.6 → P7.0 Notify & Alert	notification_type: OVERDUE, recipient_ids[], invoice_ids[]

5. Naming Conventions & Standards

The following conventions are applied consistently throughout the ProClinic DFDs and this data dictionary:

Convention	Rule Applied
Process IDs	Format: P<level>.<number> e.g., P1.0, P3.2, P4.6. Sub-processes use the parent ID as prefix.
Data Store IDs	Format: D<number> e.g., D1: Users, D10: Doctor Unavailability. Consistent across all DFD levels.
Duplicate Elements	Marked with asterisk (*) e.g., 'Doctor *', 'D3: Appointments *'. Same element, different location for clarity.
Flow Names	Descriptive noun phrases e.g., 'Login Credentials', 'Update Status: COMPLETED'. Unique within a process.
Status Values	Appointment: SCHEDULED | CHECKED_IN | COMPLETED | CANCELLED. Invoice: UNPAID | PARTIAL | PAID.
Error Flows	Rendered as dashed red arrows in DFDs e.g., 'Invalid Credentials Error', 'Conflict Error / Suggested Slots'.

Convention	Rule Applied
Reference Processes	Blue-coloured boxes in Level 2 DFDs indicating a cross-reference to another Level 2 process (e.g., P5.0, P6.0).
Data Types	String, Integer, Decimal, Boolean, Date, DateTime, Enum{}, Array[] notation used in composition column.

6. Summary

Element Category	Count	Notes
External Entities	6	Admin, Receptionist, Doctor, Patient, Pharmacist, Accountant
Data Stores	10	D1: Users through D10: Doctor Unavailability
Data Flows (Total)	87	Across Level 0, Level 1, and Level 2 DFDs
Level 2 Processes	5	P1.0, P3.0, P4.0, P5.0, P6.0 decomposed
Sub-processes Total	31	1.1–1.5, 3.1–3.7, 4.1–4.7, 5.1–5.6, 6.1–6.6